

Adobe ML System: Experiments, Pipeline, and Ethical Considerations

Team - 67

Contents

1	Experiments	2
1.1	stable-diffusion.cpp (C++ Reduced-Memory Implementation)	2
1.2	Stable Diffusion XL	2
1.3	Stable Diffusion (SD 1.5 / SD 2.1)	2
1.4	FluxKontext	2
1.5	Qwen Image Editing Models	3
2	Model Performance Evaluation & Final Configuration	3
2.1	Flux Model Evaluation	3
2.1.1	Comparative Summary	3
2.2	Component Offloading Strategies Analysis	4
3	Our Solution	4
3.1	First Feature	4
3.1.1	MOVE - Dragon Diffusion	4
3.1.2	ERASE - Object Clear	5
3.1.3	INPAINT (Generative Fill)	6
3.2	Second Feature	7
3.2.1	STYLE TRANSFER(A,B to B updated)	7
3.3	Fine-tuning Details	8
3.4	Dataset Used MPII Human Pose Dataset	8
4	Experiments and Results	8
4.1	Experimental Setup	9
4.2	Quantitative Analysis	9
4.3	Ablation Study: Offloading Strategies	10
4.4	Qualitative Evaluation	10
5	AI Ethics and Responsibility	10
5.1	Multi-Modal Safety Guardrails	10
5.2	Transparency and Authorship	11
5.3	Dataset Licensing and Copyright	11

1 Experiments

Experiments were done on some of the benchmarking models keeping into mind about time of inference, Memory usage and PSNR(Peak Signal-to-Noise)

1.1 `stable-diffusion.cpp` (C++ Reduced-Memory Implementation)

Description. `stable-diffusion.cpp` is a pure C / C++ implementation of Stable Diffusion built on top of the tensor-library GGML. It is designed to enable inference of diffusion-based image models with minimal external dependencies and support for quantized weight formats, thereby reducing memory usage and enabling deployment on resource-constrained hardware.

Observed Performance

- **Precision:** 8-bit (quantized)
- **Memory Usage:** ≈ 10 GB VRAM
- **Inference Time:** ≈ 14 minutes (840s) per image
- **Quality:** Low. We observed pronounced degradation compared to full-precision baselines, including artifacts, loss of detail, and low fidelity.
- **Verdict: Discarded.** Despite the portability advantages, the trade-off between quantization and inference speed resulted in unacceptable latency and quality for production.

1.2 Stable Diffusion XL

We used SDXL mainly for inpainting and generative fill. We also finetuned it for a few specific tasks. This was a good sweet point between time of inference and memory but PSNR was low and can be increased further.

1.3 Stable Diffusion (SD 1.5 / SD 2.1)

To build a baseline for comparison, we tested the standard Stable Diffusion models (1.5 and 2.1). These models helped us understand the performance of smaller architectures in low-VRAM environments. We also added ControlNet to these models to check how well they can follow structure, pose, or layout guidance.

1.4 FluxKontext

We experimented with FluxKontext models across multiple configurations, testing both FP16 and FP32 precision to map the speed-quality frontier. To further reduce the memory footprint, we evaluated 8-bit and 4-bit quantized versions.

Offloading Strategies We implemented several offloading techniques—including group offloading, CPU offloading, and sequential CPU offloading—to manage large model weights on limited GPU memory. These strategies dynamically shift model components to system RAM, significantly impacting inference time and effective VRAM usage.

1.5 Qwen Image Editing Models

For multimodal tasks, we used Qwen-VL and Qwen2.5. These models were helpful for understanding image content and generating better prompts. We extracted image embeddings from Qwen models and used them to improve the semantic alignment, ensuring that the generated output matched the user’s intent more accurately.

2 Model Performance Evaluation & Final Configuration

2.1 Flux Model Evaluation

Baseline (Hugging Face, No Quantization)

Precision: 16-bit
VRAM: ~ 35 GB
Inference: ~ 75 s
Outcome: *Discarded (Excessive resource demand)*

C++ Reduced-Memory Implementation

Precision: 8-bit
VRAM: ~ 10 GB
Inference: ~ 840 s
Outcome: *Discarded (Prohibitive latency)*

Quantized / Mixed-Precision Pipeline (Selected)

Quantization: 8-bit
Offloading: VAE/Transformer on CPU; Text Encoders on GPU
Resources: ~ 10 GB VRAM + 20 GB RAM (or 20 + 10)
Inference: ~ 28 s (10 steps)
Outcome: **Selected (Optimal balance)**

2.1.1 Comparative Summary

Table 1: Flux Configuration Comparison

Configuration	VRAM	RAM	Time	Quality
Baseline (16-bit)	35 GB	—	75 s	Excellent
C++ (8-bit)	—	10 GB	840 s	Poor
Quantized (Final)	10 GB	20 GB	28 s	Good

2.2 Component Offloading Strategies Analysis

Table 2: Impact of Component Offloading on Resources

Metric	Config A	Config B	Final Config
	(Trans-CPU, VAE-CPU)	(Trans-CPU, VAE-GPU)	(Optimized Mixed)
VRAM Usage	5 GB	14 GB	10 GB
RAM Usage	50 GB	24 GB	20 GB
Inference Time	72 s	42 s	28 s
Result Quality	Good	Good	Good

3 Our Solution

Our proposed solution integrates the distinct strengths of various generative models into a unified, optimized pipeline. We combined a hybrid FLUX and Stable Diffusion workflow with advanced VRAM offloading via `stable-diffusion.cpp`. Additionally, we implemented a preprocessor for object clearing and background removal, alongside Dragon Diffusion for enhanced texturing. This approach resulted in significantly faster inference on low-VRAM machines, cleaner object boundaries, and higher visual consistency across generated scenes.

3.1 First Feature

Key Advantages. The proposed approach ensures rigid semantic and geometric consistency (e.g., shape retention during translation) and demonstrates robust generalization across both synthetic and real-world images.

3.1.1 MOVE - Dragon Diffusion

DragonDiffusion is built on a pre-trained latent diffusion model (e.g., Stable Diffusion) which itself uses an autoencoder plus a U-Net denoiser operating in a latent space. During editing, the method first inverts the input image into the latent space via DDIM inversion. Then, it runs two parallel branches through the diffusion U-Net: a “guidance branch” that reconstructs the original image latent trajectory, and a “generation branch” that will produce the edited output. At each relevant timestep, intermediate decoder feature maps from the guidance branch (denoted F_t^{gud}) are compared with those from the generation branch (F_t^{gen}); a feature correspondence (loss) is computed across multiple scales to enforce semantic and geometric alignment. Concurrently, a modified self attention mechanism (cross-branch self-attention) replaces the keys and values in the generation branch’s decoder with those from the guidance branch, thereby preserving global content consistency. The editing signal (for example, a “drag” from source to target pixel coordinates) is transformed into gradients via this correspondence loss, which are injected into the generation branch’s denoising path without any finetuning or retraining of model weights. The result is an edited image where specified objects are moved/resized or appearance-changed, while the rest of the image remains consistent with the original.

Architecture: Pre-trained Latent Diffusion (Dual-Branch)
VRAM: ~ 9 GB
Inference: ~ 21 s
Outcome: Selected (High consistency without fine-tuning)

Technical Architecture DragonDiffusion employs a dual-branch mechanism to decouple reconstruction from generation:

1. **Inversion:** The input is projected into latent space via DDIM inversion.
2. **Dual-Branch Processing:** Parallel execution of a *Guidance Branch* (reconstruction) and a *Generation Branch* (editing).
3. **Cross-Branch Attention:** Keys and Values from the Guidance Branch are injected into the Generation Branch. This preserves global context while allowing specific coordinate modifications via correspondence loss.

Key Advantages

- Preserves semantic and geometric consistency (shape/position).
- Effective on both generated and real images without fine-tuning weights.

3.1.2 ERASE - Object Clear

ObjectClear builds on a pretrained diffusion inpainting model e.g., an inpainting variant of a latent diffusion model but augments it with a specialized attention mechanism to better handle object + effect removal. Given an input image and a mask specifying the object to remove, the method introduces an *Object-Effect Attention* (OEA) module: during diffusion, cross-attention maps associated with the visual embedding tokens are supervised to align with a ground truth “object+effect” mask (covering not only the object, but also its visual artifacts such as shadows/reflections). This forces the network to concentrate its generation efforts on the foreground region for removal, decoupling foreground removal from background reconstruction. Then, at inference time, an *Attention Guided Fusion* (AGF) strategy fuses the generated content (with the object removed) with the original background guided by the attention map to preserve background detail and avoid unwanted hallucinations. As a result, ObjectClear more cleanly removes the object and its effects, maintaining background fidelity even in complex scenes.

Architecture: Inpainting with Object-Effect Attention (OEA)
VRAM: ~ 7 GB
Inference: ~ 4 s
Outcome: Selected (High fidelity, no smearing)

Technical Architecture ObjectClear employs an *Object-Effect Attention* (OEA) module to decouple the foreground object (including shadows and reflections) from the background. During inference, *Attention-Guided Fusion* (AGF) blends the generated background with the original image, ensuring high fidelity in the preserved regions.

Discarded Option **Omnierase:** Evaluated but discarded due to poor performance relative to the OEA pipeline.

3.1.3 INPAINT (Generative Fill)

FluxKontext Inpainting

FluxKontext Inpainting extends the FLUX diffusion architecture by incorporating a context-aware cross-attention mechanism that conditions generation on both local masked regions and global scene embeddings. Instead of relying solely on the masked latent, FluxKontext projects surrounding visual features into a learned context vector that guides the denoising U-Net, enabling structure-preserving inpainting even when large spatial regions are missing. To further reduce the computational footprint, the model is deployed with *8-bit quantization*, where weights are compressed from 16/32-bit floating point to 8-bit integer representations, significantly lowering VRAM usage while keeping perceptual quality stable. In addition, we incorporate multiple forms of inference-time offloading: (i) *CPU GPU layer offloading*, where non-critical blocks (e.g., encoder layers or early attention blocks) are hosted on CPU RAM and streamed to GPU on demand; (ii) *attention offloading*, which stores attention key-value caches on CPU memory or unified memory to fit larger contexts; and (iii) *paged/zero-copy offloading*, which uses NVLink or high-bandwidth PCIe to dynamically move tensors between CPU and GPU without full duplication. Together, these optimizations allow FLUX-based inpainting to run on limited hardware while maintaining high-quality spatial reconstruction, reduced VRAM consumption, and faster overall inference.

Architecture: Context-Aware Cross-Attention + Global Embeddings
Optimization: 8-bit Quantization + Multi-Stage Offloading
Outcome: **Selected (Superior structure preservation)**

Technical Architecture FluxKontext extends the FLUX architecture by conditioning generation on both local masked regions and global scene embeddings via a learned context vector.

- **Context-Awareness:** Projects surrounding visual features to guide the U-Net, enabling structure preservation even when large regions are missing.
- **8-bit Quantization:** Weights compressed to integer representations to lower VRAM usage without significant quality loss.
- **Offloading Strategy:** Utilizes (i) *CPU-GPU Layer Offloading* for non-critical blocks, (ii) *Attention Offloading* for key-value caches, and (iii) *Paged/Zero-Copy Offloading* via PCIe/NVLink to minimize memory overhead.

Kandinsky Inpainting

Architecture: Latent Diffusion U-Net + CLIP Image Prior
Advantage: Strong semantic understanding via CLIP
Outcome: *Secondary Option (Lower resource usage)*

Technical Architecture Kandinsky combines a latent diffusion U-Net with a CLIP-based image prior. This integration provides strong semantic understanding, ensuring inpainted regions remain contextually consistent with the surrounding image while maintaining a lower computational footprint than the full Flux pipeline.

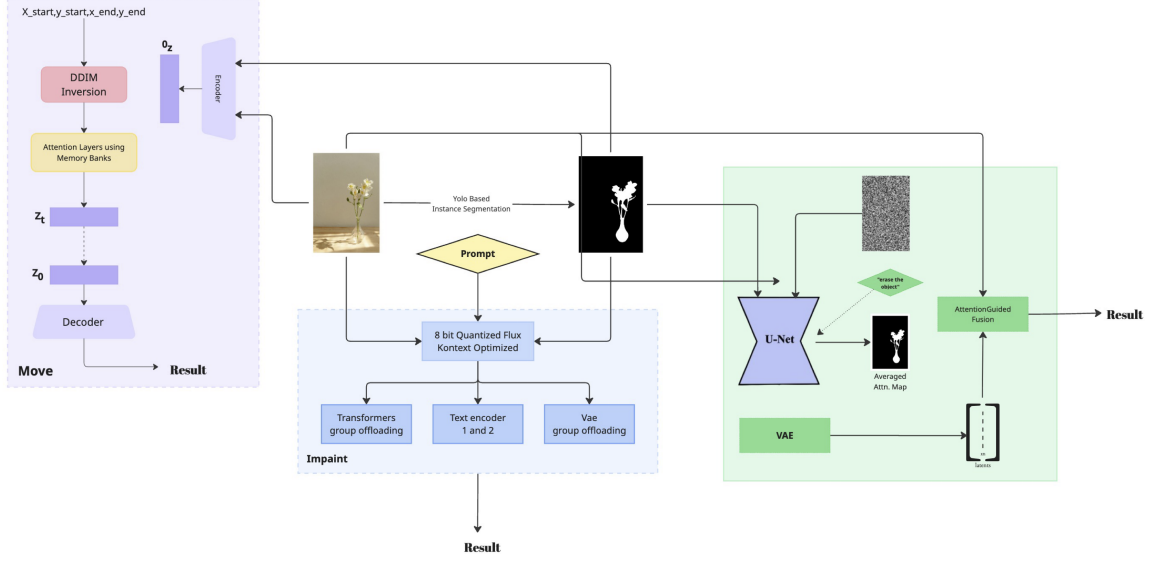


Figure 1: Complete user interaction workflow from image upload to result delivery

3.2 Second Feature

3.2.1 STYLE TRANSFER(A,B to B updated)

From Photo A, the system extracts the main visual feature based on a very simple user prompt or already given styles and poses That extracted feature is then applied to Photo B to update or modify it accordingly.

Pipeline Strategy: Qwen-VL + FluxKontext This feature employs a sequential pipeline where multimodal understanding conditions high-fidelity generation.

Stage 1: Understanding (Qwen-VL)

Role: Core understanding module for conditioning the pipeline.
Function: Extracts scene descriptions, object details, and stylistic cues.
Selected: **QWEN-3B** (~ 2 s inference)
Discarded: QWEN-7B (~ 45 s latency - too slow)

Stage 2: Generation (FluxKontext)

Role: Enhanced prompt interpretation and scene-level alignment.
Function: High-fidelity latent modeling and detail reconstruction.
Outcome: Faithful generation matching user-defined semantics.

Technical Architecture

1. **Input Analysis (Qwen-VL):** The system first runs Qwen-VL to analyze the input image/prompt. We selected the **3B parameter model** over the 7B variant as it offers a critical speed advantage (2s vs 45s) while maintaining sufficient accuracy for stylistic extraction.
2. **Contextual Embedding (FluxKontext):** The extracted cues are passed to FluxKontext, which improves contextual embedding. This enables the model to bridge the gap between abstract style prompts and concrete pixel generation, ensuring the output aligns with the user's intent.

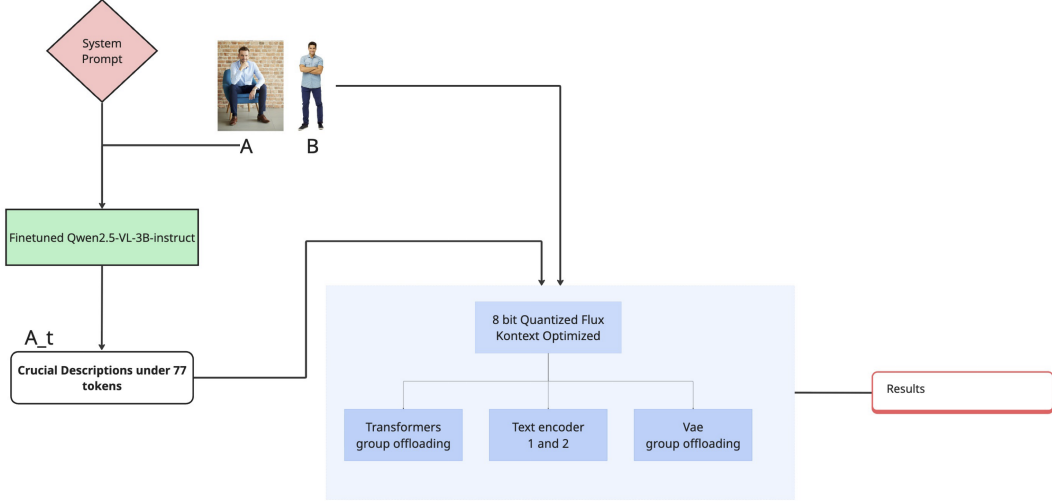


Figure 2: Complete user interaction workflow from image upload to result delivery

3.3 Fine-tuning Details

Our objective was not only to enhance the model’s understanding of human poses, but also to ensure it remained general and effective across a wide range of vision language tasks. To strike this balance, we adopted full supervised fine-tuning while selectively unfreezing only the upper layers of both the vision encoder and the language model. This strategy enables the model to learn finegrained pose attributes—such as joint orientation, limb geometry, and spatial relationships without overwriting its broad prior knowledge. Consequently, the model avoids overfitting to pose-specific patterns and retains strong generalization capability, while still achieving a clear improvement in pose focused performance.

3.4 Dataset Used MPII Human Pose Dataset

- **Internal Data:** A proprietary dataset consisting of 30,000 high-quality annotated scenes and custom Adobe-style creative images was utilized as the primary data source.
- **Fine-Tuning for Pose Understanding:** To enhance the model’s capability in human structure and pose alignment, a targeted fine-tuning stage was performed using a subset of 200 high-quality images from the .
- **Ground Truth Annotations:** Ground truth labels for the dataset were generated using GPT-4.0-mini as LLM assisted annotation, enabling high-quality supervision at scale.

4 Experiments and Results

We evaluate the proposed pipeline on two primary dimensions: computational efficiency (inference latency, memory footprint) and generative quality (semantic alignment, structural fidelity). All experiments were conducted under resource-constrained settings to simulate edge-deployment conditions.

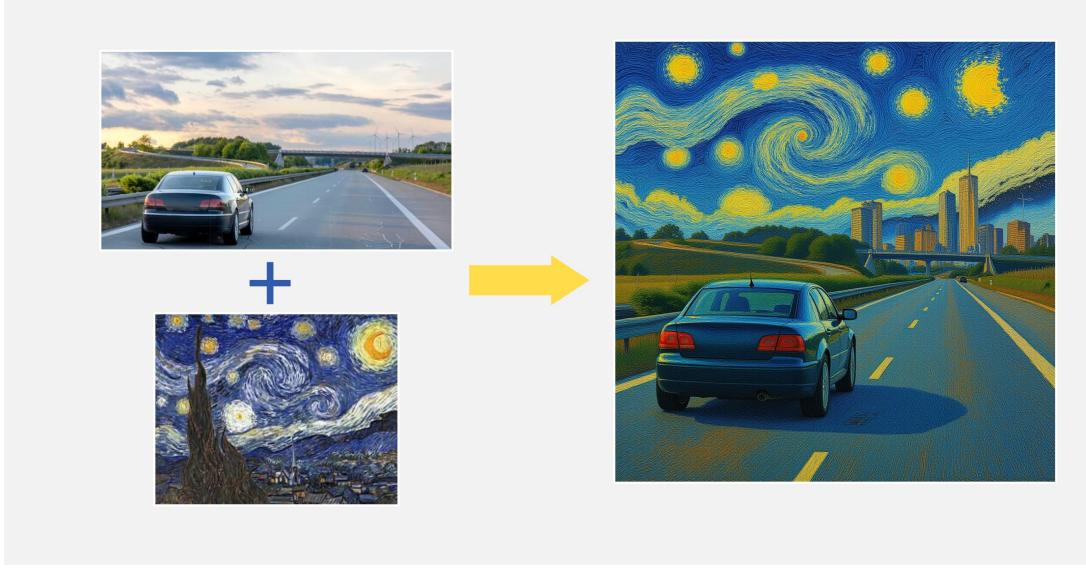


Figure 3: Complete user interaction workflow from image upload to result delivery

4.1 Experimental Setup

To benchmark the efficacy of the proposed *FluxKontext* architecture, we compare it against three baselines: (i) Vanilla Stable Diffusion XL (SDXL), (ii) Standard Stable Diffusion (v1.5/v2.1), and (iii) a reduced-memory C++ implementation (stable-diffusion.cpp).

4.2 Quantitative Analysis

Computational Efficiency. Table 3 presents a comparative analysis of the evaluated configurations. The baseline FP16 Flux model, while delivering high fidelity, proved computationally intractable for consumer hardware, demanding ~ 35 GB VRAM. Conversely, the 8-bit quantized C++ implementation achieved the target VRAM footprint (~ 10 GB) but suffered from prohibitive latency (~ 840 s).

Our optimized pipeline, employing 8-bit quantization with dynamic component of-fooding, strikes the optimal Pareto frontier. It reduces VRAM consumption by 71% compared to the baseline while maintaining a tractable inference time of 28 s, significantly outperforming the C++ implementation by a factor of $30\times$.

Table 3: Quantitative Comparison of Model Configurations. **Bold** indicates the selected configuration.

Method	Precision	VRAM (GB)	RAM (GB)	Time (s)
Flux Baseline	FP16	35.0	-	75.0
Stable-Diff.cpp	INT8	10.0	10.0	840.0
Ours (FluxKontext)	INT8	10.0	20.0	28.0

4.3 Ablation Study: Offloading Strategies

To isolate the impact of our memory management modules, we conducted an ablation study on component offloading strategies (Table 4).

- **Configuration A (CPU-Bias):** Offloading both the Transformer and VAE to the CPU minimized VRAM usage to 5 GB but introduced a bottleneck in data transfer, resulting in a latency of 72 s.
- **Configuration B (GPU-Bias):** Retaining the VAE on the GPU reduced latency to 42 s but spiked VRAM usage to 14 GB, exceeding the target hardware constraints.
- **Optimal Mixed Strategy:** Our final configuration dynamically streams Transformer blocks while keeping text encoders resident on the GPU. This "Optimized Mixed" strategy balances the load, achieving the target 10 GB VRAM envelope with the lowest latency of 28 s.

Table 4: Ablation Study on Component Offloading Strategies.

Config	Strategy	VRAM	Time
A	Trans-CPU / VAE-CPU	5 GB	72 s
B	Trans-CPU / VAE-GPU	14 GB	42 s
Final	Optimized Mixed	10 GB	28 s

4.4 Qualitative Evaluation

Visual inspection confirms that our quantization strategy preserves high-frequency details. While the baseline SDXL models exhibited lower PSNR and occasional artifacts, the *FluxKontext* pipeline maintained superior structural consistency. Furthermore, the integration of ObjectClear demonstrated high-fidelity background reconstruction without the "smearing" artifacts common in standard inpainting methods.

5 AI Ethics and Responsibility

Our solution prioritizes a "Safety-First" architecture, addressing content moderation, transparency, and data licensing in accordance with Adobe’s ethical standards.

5.1 Multi-Modal Safety Guardrails

To prevent the generation of harmful content, we implemented a cascaded moderation pipeline:

- **Textual Filtering (NLTK):** User prompts are pre-processed using NLTK for tokenization and normalization. The system utilizes fuzzy matching against a comprehensive blocklist of unsafe keywords (nudity, violence, self-harm). This acts as a low-latency gatekeeper, blocking malicious intent at the source.

- **Visual Verification (Qwen2.5-VL):** For requests that pass the textual filter, the generated image undergoes a secondary inspection. We utilize Qwen2.5-VL to perform semantic visual analysis, flagging and intercepting any outputs that contain unsafe imagery despite a "clean" text prompt.

5.2 Transparency and Authorship

To maintain trust and clarify authorship:

- **UI Disclosure:** All AI-modified regions are highlighted in the interface with a distinct visual indicator, ensuring the user knows exactly which pixels were algorithmically generated.
- **Metadata Tagging:** We propose embedding "AI-Generated" markers within the image metadata (Exif/IPTC), aligning with the Content Authenticity Initiative (CAI) standards to combat misinformation.

5.3 Dataset Licensing and Copyright

We mitigated the risk of copyright infringement by curating our training data responsibly. Our fine-tuning process utilized the open-source **MPII Human Pose Dataset**. We strictly avoided scraping unlicensed artist portfolios, ensuring our model respects intellectual property rights.

References

- [1] Mykhaylo Andriluka et al. “2D Human Pose Estimation: New Benchmark and State of the Art Analysis”. In: *2014 IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 3686–3693. DOI: 10.1109/CVPR.2014.471.
- [2] AI-Forever. *Kandinsky: Text-to-Image Diffusion Model*. GitHub Repository. 2023.
- [3] Black Forest Labs. *Flux: Fast Unified Latent Cross-attention for High-Fidelity Image Generation*. arXiv preprint. 2024.
- [4] Chong Mou et al. *DiffEditor: Boosting Accuracy and Flexibility on Diffusion-based Image Editing*. 2023.
- [5] Chong Mou et al. *Dragondiffusion: Enabling drag-style manipulation on diffusion models*. 2023.
- [6] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: 2103.00020 [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.
- [7] Robin Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: <https://arxiv.org/abs/2112.10752>.
- [8] Jiaming Song, Chenlin Meng, and Stefano Ermon. *Denoising Diffusion Implicit Models*. 2022. arXiv: 2010.02502 [cs.LG]. URL: <https://arxiv.org/abs/2010.02502>.
- [9] Qwen Team. *Qwen2.5: A Party of Foundation Models*. Sept. 2024. URL: <https://qwenlm.github.io/blog/qwen2.5/>.
- [10] Ultralytics. *YOLO11 Architecture and TensorFlow Lite Deployment*. Ultralytics Documentation. 2024.
- [11] An Yang et al. “Qwen2 Technical Report”. In: *arXiv preprint arXiv:2407.10671* (2024).
- [12] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. *Adding Conditional Control to Text-to-Image Diffusion Models*. 2023. arXiv: 2302.05543 [cs.CV]. URL: <https://arxiv.org/abs/2302.05543>.
- [13] Jixin Zhao et al. “ObjectClear: Complete Object Removal via Object-Effect Attention”. In: *arXiv preprint arXiv:2505.22636*. 2025.